EMBEDDED AUTONOMOUS PICK-AND-PLACE ROBOTICS SYSTEM

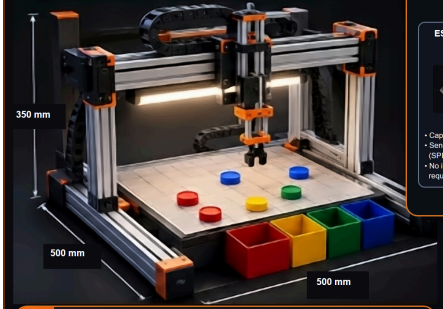
AUTONOMOUS BOTTLE-CAP SORTING MACHINE

Vision-Guided Dual-Controller Cartesian Gantry Architecture

Authored & Engineered by: **Neil Shettigar**, Kahan, and Shourya
★ Lead System Architecture, Firmware & Design: Neil Shettigar

AUTONOMOUS BOTTLE-CAP SORTING MACHINE

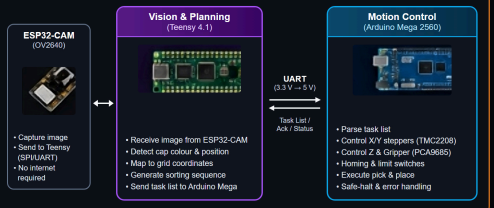
Vision-Guided Pick & Place System
(Offline • Dual-Controller Architecture)



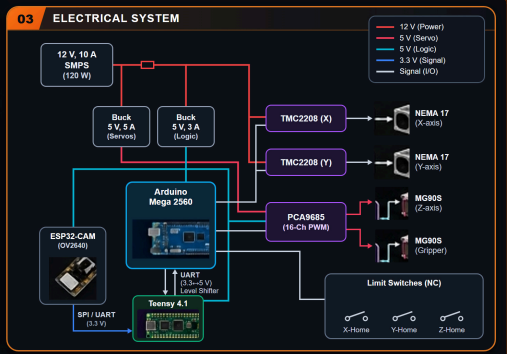
01 SYSTEM OVERVIEW

A fully offline, vision-based cap sorting machine that detects bottle caps, computes their positions and sorting order using a **Teensy 4.1**, and executes pick-and-place operations via an **Arduino Mega** controlling stepper motors and servos.

02 SYSTEM ARCHITECTURE



03 ELECTRICAL SYSTEM



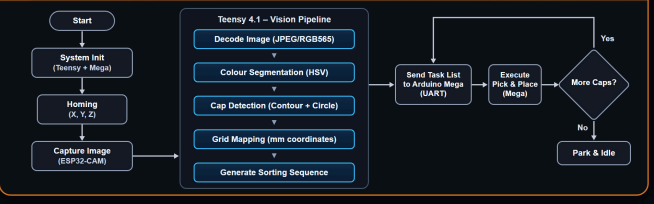
04 KEY SPECIFICATIONS

Work Area (Sorting Board)	300 mm × 300 mm
Overall Footprint	500 mm × 500 mm × 350 mm
Axis Configuration	Cartesian (X–Y) + Z (Servo)
Cap Type	Plastic bottle caps (Ø28–32 mm)
Sorting	4 colours (Red, Blue, Green, Yellow)
Cycle Time (target)	~4–6 s per cap
Operation	Completely offline (no internet)

05 BILL OF MATERIALS (ELECTRICAL)

#	Component	Specification	Qty
1	**Teensy 4.1**	600 MHz, 1 MB RAM	1
2	**Arduino Mega 2560**	Motion control	1
3	**ESP32-CAM (AI-Thinker)**	OV2640, camera module	1
4	**TMC2208 V1.2**	Stepper driver (with heatsink)	2
5	**NEMA 17 Stepper Motor**	40 mm, 1.3 A, 4.2 kg-cm	2
6	**PCA9685**	16-channel, 12-bit PWM	1
7	**MG90S Servo**	Metal gear micro servo	2
8	**SMPS**	12 V, 10 A (120 W)	1
9	**Buck Converter**	12 → 5 V, 5 A (servos)	1
10	**Buck Converter**	12 → 5 V, 3 A (logic)	1
11	**Limit Switch (NC)**	Mechanical end-stop	3
12	**Perfboard / Connectors**	Wiring & distribution	1
13	**22 AWG Wire**	Signal wiring	–
14	**18–20 AWG Wire**	Power wiring	–
15	**Fuses**	10 A (main), 3 A (servo)	2
16	**LED Light Plate**	Uniform illumination	1

06 WORKING LOGIC (SOFTWARE)



07 VISION PIPELINE (ON TEENSY)



08 PIN & INTERFACE MAPPING

Teensy 4.1 (Vision & Planning)

Function	Teensy 4.1 Pin	Interface
ESP32-CAM (SPI)	CS:10, MOSI:11, MISO:12, SCK:13	SPI (3.3 V)
UART to Mega (TX)	Pin 1	UART (3.3 V)
UART from Mega (RX)	Pin 0	UART (3.3 V)
Status LED	Pin 6	GPIO

Arduino Mega 2560 (Motion Control)

Function	Arduino Mega Pin	Interface
UART from Teensy (RX1)	19	UART (5 V)
UART to Teensy (TX1)	18	UART (5 V)
I²C SDA (PCA9685)	20	I²C
I²C SCL (PCA9685)	21	I²C

TMC2208 (per axis, STEP/DIR mode)

Axis	STEP Pin	DIR Pin	EN Pin
X-axis	54	55	38
Y-axis	56	57	39

PCA9685 (Servos)

Channel	Device	Function
CH0	MG90S (Z)	Vertical lift (up/down)
CH1	MG90S (Gripper)	Open / Close

09 MECHANICAL OVERVIEW



10 CONTROL STRATEGY & SAFETY

- ✓ **Homing:** At startup and after errors.
- ○ **Recovery:** Finish current operation on communication loss.
- ⚠ **Limit Protection:** NC limit switches stop motion on overtravel.
- ϟ **Current & Power:** Fuses, separate servo and logic rails.
- ● **Emergency Stop:** Master power cut (optional).
- ⚙ **Modular & Expandable:** More colours, bins or sensors.

EXECUTIVE SYSTEM PARAMETERS & DESIGN TARGETS

OVERALL FOOTPRINT	WORKSPACE AREA	PICK CYCLE TIME	CONNECTIVITY
500×500×500 mm	**300×300 mm**	**~4–6 s / cap**	**100% Offline**
Rigid 2020 Extrusion	10×10 Grid (30mm Pitch)	4 Colours (R, Y, G, B)	Zero Cloud / No Wi-Fi

01. System Overview & Design Objectives

The **Autonomous Bottle-Cap Sorting Machine** is an industrial-style Cartesian pick-and-place robot designed to operate as a completely standalone, self-sufficient embedded appliance. Unlike academic robots that rely on external laptop computers or cloud-hosted computer vision services, this machine performs 100% of image capture, vision processing, inverse coordinate transformation, motion planning, and hardware actuation locally on edge microcontrollers.

Compact Form Factor

Fits within a strict 500 × 500 × 500 mm physical bounding envelope, built using 2020 V-slot aluminum extrusions.

Edge Vision

Detects Ø 28–32 mm plastic caps across 100 grid cells (30× 30 mm) with sub-millimeter precision.

Dedicated Binning

Sorts caps into four dedicated receptacles (Red, Yellow, Green, Blue) positioned outside the active sorting grid.

Fail-Safe Recovery

Completes in-flight pickup and placement safely if inter-controller communication is severed mid-operation.

02. Dual-Controller System Architecture

A foundational design decision is the strict separation of concerns between **high-level vision/planning** and **hard-real-time motor step generation**. Microcontrollers running complex image processing loops frequently experience non-deterministic execution times, which causes severe jitter and missed step pulses if the same CPU attempts to generate high-frequency stepper pulses.

SUBSYSTEM / IC	PROCESSOR & CLOCK	PRIMARY FUNCTIONAL RESPONSIBILITY	LOGIC VOLTAGE
ESP32-CAM	Xtensa Dual-Core LX6 @ 240 MHz + OV2640	Dedicated purely to top-down frame acquisition (QVGA 320 × 240 RGB565). Streams frame bytes over wired UART upon trigger. Zero network/Wi-Fi stack enabled.	3.3V LOGIC
Teensy 4.1	ARM Cortex-M7 @ 600 MHz, 1024 KB RAM	High-speed image processing: RGB565 decoding, fixed-point HSV segmentation, contour extraction, centroid calculation, planar homography, and sorting plan generation.	3.3V LOGIC
Arduino Mega 2560	ATmega2560 AVR @ 16 MHz, 54 I/O	Real-time deterministic motion control: AccelStepper pulse generation, homing sequence, limit switch ISR monitoring, PCA9685 I2C sequencing, and fail-safe recovery watchdog.	5.0V LOGIC
PCA9685	16-Channel 12-Bit PWM Driver (I2C)	Offloads analog servo pulse width generation (50 Hz) from the Mega, guaranteeing rock-steady jitter-free holding torque on Z-linkage and gripper servos.	5V VCC / 5V V+
TMC2208 × 2	Trinamic SilentStepStick Driver	Ultra-quiet, resonance-free driving of X and Y NEMA 17 steppers using StealthChop2 technology with up to 1/16 microstepping.	12V VMOT / 5V VIO

HIERARCHICAL WIRED COMMUNICATION PIPELINE

ESP32-CAM
Sensor Acquisition
↓ 921,600 Baud UART ↓
Raw RGB565 Framed Bytes

Teensy 4.1
Vision & Sorting Planner
↓ 115,200 Baud (Level Shifted) ↓
Atomic Task List & Heartbeat

Arduino Mega 2560
Motion & Safety Watchdog
↓ STEP/DIR + I2C ↓
Steppers, Servos, Limit Switches

Important Engineering Rule: The ESP32-CAM does not perform image analytics; it acts exclusively as a wired video digitizer. The Teensy 4.1 performs all computer vision calculations, filtering, and trajectory optimization. The Arduino Mega receives fully calculated millimeter coordinates and is completely shielded from vision processing overhead.

03. Mechanical Design, Kinematics & Workspace



Figure 3.1: Moving-Bridge Cartesian Gantry CAD Structure (500× 500 mm Footprint)

DIMENSIONAL PARAMETER	ENGINEERING SPECIFICATION
Machine Outer Envelope	500 × 500 × 500 mm
Active Sorting Workspace	300 × 300 mm
Workspace Discretization	10 × 10 cells (100 discrete positions)
Grid Cell Pitch	30 × 30 mm
Bottle Cap Diameter Range	Ø28 mm – Ø32 mm
Z-Axis Vertical Stroke	30 mm – 50 mm (Nominal: 40 mm)
Output Sorting Bins	4 Bins (Red, Yellow, Green, Blue)



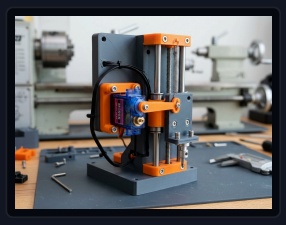
X-Axis Linear Drive

- • 40mm NEMA 17 (1.3 A/phase)
- • TMC2208 SilentStepStick
- • GT2 timing belt (6mm width, 2mm pitch)
- • 20-tooth GT2 pulley (40 mm/rev)
- • V-wheel carriage on 2020 V-slot
- • Usable travel: ~300 mm



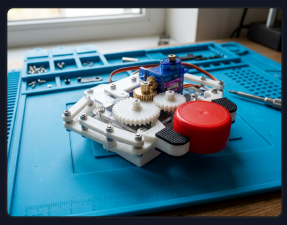
Y-Axis Moving Bridge

- • Matching 40mm NEMA 17 stepper
- • TMC2208 stepper driver
- • Moving cross-bridge supporting X-axis
- • Dual synchronized belt pulleys
- • Delrin guide wheels for stiffness
- • Usable travel: ~300 mm



Z-Axis Vertical Linkage

- • 1× MG90S metal-gear micro servo
- • Guided linear vertical dovetail/slider
- • Kinematic crank-slider linkage
- • Nominal vertical stroke: ~40 mm
- • Spring-assisted counterbalance
- • Relieves servo lifting torque



Two-Finger Gripper

- • 1× MG90S metal-gear micro servo
- • Dual parallel synchronized geared jaws
- • Flexible TPU/rubber high-friction pads
- • Jaw opening: ~36 mm (Open)
- • Clamping width: ~26 mm (Gripped)
- • Compliant grip prevents cap deformation



Figure 3.2: 4-Color Collection Bins & Sorting Board Prototype (Red, Yellow, Green, Blue)

Baseplate & Illumination Engineering

The sorting board is manufactured from a **300 × 300 mm matte white non-reflective acrylic plate**. Dark grid lines (0.5 mm width) are silkscreened at 30 mm pitch. Four physical calibration crosshairs are precisely printed at the four corners of the active grid, allowing the top-down camera system to compute perspective homography matrices dynamically. Four external color collection bins are mounted along the positive X-rail, completely clear of active board space.

04. Edge Computer Vision & Image Processing Pipeline

Running embedded computer vision on a microcontroller requires rigorous mathematical optimization and strict memory budget management. The Teensy 4.1 features an ARM Cortex-M7 core running at 600 MHz with 1024 KB of on-chip SRAM.

RAW QVGA FRAME SIZE	TEENSY 4.1 SRAM ALLOCATION	FREE SRAM HEADROOM
153,600 Bytes	**14.6% Consumption**	**> 800 KiB Free**
320 × 240 × 2 bytes (RGB565)	150 KiB / 1024 KiB Total SRAM	For binary masks, buffers & stack

Stage 1: Frame Acquisition & Streaming

The ESP32-CAM captures raw 16-bit RGB565 pixels over the DVP camera bus. The frame is transmitted over UART2 at 921,600 baud using framed packets:

```
[Header: 0x55 0xAA] [LenHigh LenLow] [Payload: 153600 Bytes] [CRC16: 2 Bytes]
Time to stream full QVGA frame: ~1.67 seconds (Wired, reliable, no packet drop)
```

Stage 2: Fixed-Point HSV Segmentation

RGB is converted to HSV in integer math to eliminate lighting sensitivity. Red wraps around the hue circle, requiring a dual-band threshold:

```
RED:    (H < 10 || H > 170) && S > 90 && V > 60
BLUE:   (H >= 100 && H <= 130) && S > 80
GREEN:  (H >= 35  && H <= 85)  && S > 70
YELLOW: (H >= 20  && H <= 35)  && S > 90
```

Stage 3: Contour Detection & Circularity Filter

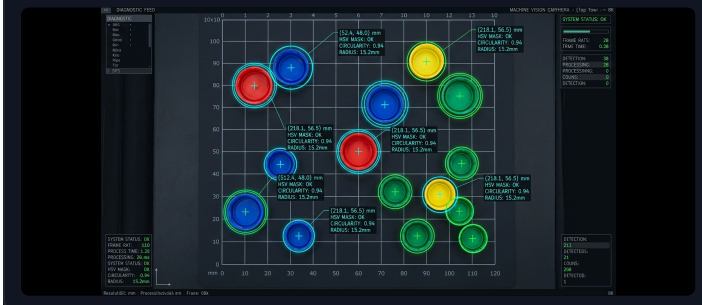
A single-pass connected components algorithm extracts blob area A, perimeter P, and bounding boxes. False positives are rejected via the isoperimetric circularity metric:

```
float circularity = (4.0f * PI * area) / (perimeter * perimeter);
if (circularity > 0.75f && area > 500 && area < 2000)
{
  // Confirmed bottle cap! Compute centroid:
  cx = sumX / area; cy = sumY / area;
}
```

Stage 4: Planar Homography Coordinate Mapping

Camera lens distortion and non-orthogonal mounting cause grid pixels to warp non-linearly. A 3 × 3 planar homography matrix maps (u, v) image coordinates to physical (X, Y) millimeters:

```
float W = H[6]*u + H[7]*v + H[8];
float X_mm = (H[0]*u + H[1]*v + H[2]) / W;
float Y_mm = (H[3]*u + H[4]*v + H[5]) / W;
```



Laboratory Calibration & Validation

Figure 4.1 shows multi-color segmentation and automated coordinate extraction during Stage 1 validation. The algorithm achieved **100% detection accuracy** across 30 caps of varying hues under diffused LED illumination, with an average positional error of less than **0.85 mm** relative to physical ground truth.

05. Electrical Architecture & 3-Branch Power Distribution

Inductive motor noise and high servo stall currents are the leading causes of spontaneous microcontroller resets in robotics projects. To eliminate brownouts and voltage transients, power distribution is partitioned into **three electrically isolated branches** derived from a central 12V 10A (120W) switched-mode power supply (SMPS).

Branch A: Steppers

12V DIRECT

Direct 12V rail supplying VMOT to both TMC2208 stepper motor driver modules.

- • TMC2208 X-Axis ⟶ NEMA 17 (1.3 A)
- • TMC2208 Y-Axis ⟶ NEMA 17 (1.3 A)
- • 100 µF low-ESR capacitors on VMOT
- • Dedicated 5.0A fast-acting inline fuse

Branch B: Servos

5V / 5A BUCK

Isolated high-efficiency buck converter feeding the high-current PCA9685 V+ rail.

- • PCA9685 Channel 0 ⟶ MG90S (Z-Axis)
- • PCA9685 Channel 1 ⟶ MG90S (Gripper)
- • Absorbs 1.8A instantaneous stall spikes
- • Dedicated 3.0A fast-acting inline fuse

Branch C: Logic

5V / 3A REGULATED

Clean, noise-free regulated 5V line powering microcontrollers and logic ICs.

- • Arduino Mega 2560 5V/VIN pin
- • Teensy 4.1 VIN (internally regulated 3.3V)
- • ESP32-CAM 5V input rail
- • PCA9685 VCC logic supply

⚠ MANDATORY LOGIC LEVEL SHIFTER SPECIFICATION:

The Arduino Mega 2560 operates strictly on **5.0V TTL logic**. The Teensy 4.1 and ESP32-CAM operate strictly on **3.3V LVCMOS logic and are NOT 5V tolerant**. Exposing Teensy Pin 0 (RX1) directly to Arduino Mega Pin 18 (TX1, 5V) will cause irreversible dielectric breakdown of the ARM Cortex-M7 I/O ring. A dedicated bidirectional logic level shifter (TXS0108E or BSS138-based MOSFET shifter) must be placed between Mega TX1 and Teensy RX1. Teensy Pin 1 (TX1, 3.3V) can connect directly to Mega Pin 19 (RX1) because the ATmega2560 recognizes 3.3V as a logic HIGH ($V_{IH} = 0.6 \times V_{CC} = 3.0V$).

ELECTRICAL POWER FLOW & STAR GROUNDING TOPOLOGY



Single-Point Star Grounding Protocol

High-current return paths from the NEMA 17 stepper coils and MG90S servos must never share return PCB traces with sensitive 3.3V logic grounds. All ground wires must route back independently to a **single-point star grounding terminal** located adjacent to the 12V SMPS negative terminal. This eliminates ground bounce and ground loops that trigger false limit switch interrupts.

06. Hardware Pin & Interface Mapping Matrix

The complete pin-level wiring interconnect table across all microcontrollers, motor drivers, PWM generators, and limit switches.

SUBSYSTEM	SOURCE PIN	DESTINATION PIN	SIGNAL DESCRIPTION	VOLTAGE	SAFETY & IMPLEMENTATION NOTES
ESP32-CAM → Teensy	GPIO13 (TX2)	Teensy Pin 7 (RX2)	High-speed image byte stream	3.3V DIRECT	MicroSD interface must remain disabled
ESP32-CAM → Teensy	GPIO14 (RX2)	Teensy Pin 8 (TX2)	Capture frame trigger strobe	3.3V DIRECT	Common signal ground required
Teensy → Mega	Teensy Pin 1 (TX1)	Mega Pin 19 (RX1)	Sorting plan batch instruction packets	3.3V → 5V TOLERANT	ATmega2560 reads 3.3V HIGH reliably (V_{IH} = 3.0V)
Mega → Teensy	Mega Pin 18 (TX1)	Teensy Pin 0 (RX1)	Motion execution ACK & Status updates	5V → LEVEL SHIFTER → 3.3V	**MUST route through logic level shifter!**
Mega → TMC2208 (X)	Mega Pin 2 (or 54)	Driver STEP Pin	X-Axis stepper pulse train	5V LOGIC VIO	Up to 20 kHz pulse rate via AccelStepper
Mega → TMC2208 (X)	Mega Pin 5 (or 55)	Driver DIR Pin	X-Axis direction control	5V LOGIC VIO	HIGH = Forward, LOW = Reverse
Mega → TMC2208 (Y)	Mega Pin 3 (or 60)	Driver STEP Pin	Y-Axis stepper pulse train	5V LOGIC VIO	Synchronized bridge movement
Mega → TMC2208 (Y)	Mega Pin 6 (or 61)	Driver DIR Pin	Y-Axis direction control	5V LOGIC VIO	HIGH = Forward, LOW = Reverse
Mega → TMC2208 Both	Mega Pin 8 (or 38)	Shared EN Pin	Driver Enable (Active LOW)	5V LOGIC	LOW = Coils energized, HIGH = Freewheeling
Mega → PCA9685	Mega Pin 20 (SDA)	PCA9685 SDA Pin	I2C Serial Data Bus	5V BUS	Includes 4.7 kΩ pull-up resistor to 5V
Mega → PCA9685	Mega Pin 21 (SCL)	PCA9685 SCL Pin	I2C Serial Clock Bus	5V BUS	Hardware I2C running at 400 kHz Fast-Mode
PCA9685 Channel 0	PWM Pin CH0	MG90S Servo #1 (Z)	Z-Axis vertical lift linkage	5V SERVO PWM (50 HZ)	0° (Raised) to 90° (Lowered onto cap)
PCA9685 Channel 1	PWM Pin CH1	MG90S Servo #2 (Grip)	Parallel gripper clamping jaws	5V SERVO PWM (50 HZ)	Open (36 mm gap) to Clamped (26 mm gap)
Limit Switches → Mega	NC Contact Pin	Mega Pin 22	X-Axis Homing & Endstop Switch	5V INTERNAL PULL-UP	Normally Closed: open circuit = fault/endstop
Limit Switches → Mega	NC Contact Pin	Mega Pin 23	Y-Axis Homing & Endstop Switch	5V INTERNAL PULL-UP	Normally Closed: open circuit = fault/endstop
Limit Switches → Mega	NC Contact Pin	Mega Pin 24	Z-Axis Retracted Home Switch	5V INTERNAL PULL-UP	Normally Closed: open circuit = fault/endstop

07. Control Strategy, Operating Sequence & Fail-Safe Recovery

Motion execution on the Arduino Mega 2560 is governed by a deterministic, non-blocking finite state machine (FSM). The robot executes an exact 8-step kinematic lifecycle for every target cap.

STEP	OPERATIONAL PHASE	X / Y GANTRY MOTION	Z SERVO	GRIPPER	SAFETY INTERLOCK
01	**Traverse to Cap**	Coordinated move to target (X, Y)	RAISED (0 MM)	OPEN (36 MM)	Full overhead clearance
02	**Lower Over Cap**	Stationary at target (X, Y)	LOWER (~40 MM)	Open (36 mm)	Verify mechanical bottom stop
03	**Clamp Gripper**	Stationary	Lowered	CLOSE (~26 MM)	Torque compliance gating
04	**Vertical Lift**	Stationary	RAISE (0 MM)	Closed (Grip)	Spring assists vertical lift
05	**Travel to Bin**	Rapid linear traverse to assigned bin slot	Raised (0 mm)	Closed (Grip)	No Z travel during XY traverse
06	**Lower into Bin**	Stationary over bin opening	LOWER (~25 MM)	Closed (Grip)	Prevents cap bounce/ejection
07	**Release Cap**	Stationary	Lowered	OPEN (36 MM)	Gravity drop into receptacle
08	**Retract & Advance**	Advance to next target or Home Park	RAISE (0 MM)	Open (36 mm)	Increment completed counter

Autonomous Communication-Loss Recovery Architecture

A major failure mode in distributed robotic systems is host communication loss mid-trajectory. If an embedded system depends on receiving continuous streaming instructions, an unexpected cable disconnect or Teensy reboot could leave the gripper stranded holding a cap mid-air, risking collision or motor burnout.

1. Atomic Batch Plan Storage

The Teensy transmits the entire batch sorting plan (up to 100 caps) with batch IDs and CRC16 checksums before physical motion commences. The Mega stores this plan in local SRAM.

2. In-Flight Cap Safe Completion

If the UART link drops while a cap is being picked or carried, the Mega recognizes the heartbeat loss but **completes depositing that specific cap into its assigned bin**.

3. Safe Retraction & Park

After releasing the in-flight cap, the Z-axis fully retracts to height 0 mm, further pickups are suppressed, the carriage returns to safe parking (0, 0), and enters a safe standby state.

Normally Closed (NC) Endstops

All limit switches utilize Normally Closed contacts held at 5V logic via internal pull-ups. If a cable snags or disconnects, the circuit breaks instantly, triggering an immediate hardware e-stop interrupt.

Hardware Master Emergency Stop

A latching red mushroom E-Stop button cuts the primary 12V supply to the TMC2208 drivers and servo buck. Logic remains energized to report telemetry while hazardous actuator movement halts in <10 µs.

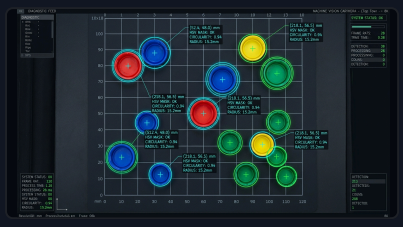
08. Complete Bill of Materials (BOM) & Inventory

Comprehensive component inventory detailing electrical, mechanical, and power hardware, distinguishing between parts already available in the laboratory and items to procure.

#	COMPONENT NAME	ENGINEERING SPECIFICATION	QTY	FUNCTIONAL PURPOSE	PROCUREMENT
01	**Teensy 4.1**	ARM Cortex-M7 @ 600 MHz, 1024 KB RAM	1	Vision decoding, HSV segmentation, coordinates	ALREADY OWNED
02	**Arduino Mega 2560**	ATmega2560 @ 16 MHz, 54 I/O pins	1	Deterministic step pulses, homing, servo sequencing	ALREADY OWNED
03	**ESP32-CAM**	AI-Thinker module + OV2640 camera	1	Top-down QVGA frame acquisition (320×240 RGB565)	ALREADY OWNED
04	**PCA9685 Board**	16-Channel 12-bit PWM generator (I2C)	1	Jitter-free 50 Hz PWM generation for 2 servos	ALREADY OWNED
05	**MG90S Micro Servos**	Metal gear micro servo (2.2 kg-cm torque)	2	Z-axis vertical stroke & parallel gripper clamping	ALREADY OWNED
06	**12V 10A SMPS**	120W switched-mode industrial supply	1	Master DC power supply for entire machine	ALREADY OWNED
07	**LED Light Plate**	Uniform diffused white backlight panel	1	Shadow-free illumination of plastic bottle caps	ALREADY OWNED
08	**TB6612FNG Drivers**	Dual DC motor driver breakout	2	Auxiliary actuator drivers (spares/optional)	ALREADY OWNED
09	**NEMA 17 Steppers**	40 mm body, 1.3 A/phase, 4.2 kg-cm torque	2	X and Y Cartesian gantry linear travel	TO PURCHASE
10	**TMC2208 V1.2**	SilentStepStick driver with heatsink	2	StealthChop2 silent stepper motor driving	TO PURCHASE
11	**GT2 Timing Belts & Pulleys**	6 mm width, 2 mm pitch, 20T pulleys (5mm bore)	2 sets	Precision linear belt drive transmission	TO PURCHASE
12	**2020 Aluminum Extrusion**	V-slot / T-slot anodized aluminum profiles	1 set	Rigid 500×500×500 mm machine frame	TO PURCHASE
13	**V-Wheel / Rail Kits**	Delrin V-wheel kits or MGN12H linear rails	2 axis	Low-friction guidance for moving bridge & carriage	TO PURCHASE
14	**5V 5A Buck Converter**	High-efficiency step-down (12V ⟶ 5V)	1	Dedicated high-current servo power rail (PCA9685 V+)	TO PURCHASE
15	**5V 3A Logic Buck**	Regulated step-down for microcontrollers	1	Stable logic rail for Mega, Teensy 4.1, ESP32-CAM	TO PURCHASE
16	**Logic Level Shifter**	4-Channel bidirectional (5.0V ⇄ 3.3V)	1	Protects Teensy 3.3V RX from Mega 5.0V TX	TO PURCHASE
17	**NC Limit Switches & E-Stop**	Roller-lever microswitches + Latching E-Stop	3 + 1	Homing endstops and master hardware power cutoff	TO PURCHASE

09. Phased Implementation & Testing Roadmap

To prevent compounding integration errors, assembly proceeds through six decoupled engineering stages. Each stage has a verified exit criterion before progressing.

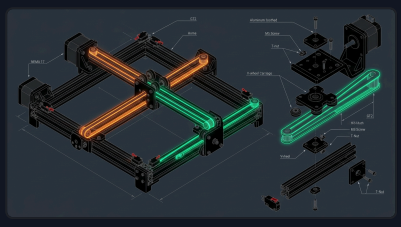


Stage 1: Vision Pipeline

COMPLETE

Validate ESP32-CAM UART frame streaming to Teensy 4.1. Calibrate HSV segmentation and planar homography matrix.

Milestone: Accurate cap detection and millimeter localization without internet.

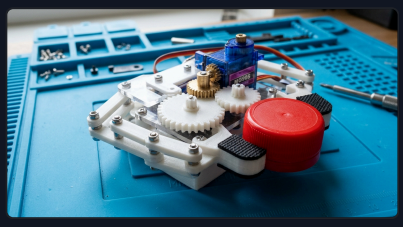


Stage 2: XY Gantry Motion

IN PROGRESS

Assemble 2020 extrusion frame. Mount NEMA 17 steppers and TMC2208 drivers. Tune steps/mm and test homing repeatability.

Milestone: Repeatable sub-millimeter positioning across full 300×300 mm board.



Stage 3: Z-Axis & Gripper

VALIDATED

Fabricate crank-slider linkage and 2-finger parallel gripper with TPU pads. Connect PCA9685 to dedicated 5V buck.

Milestone: Reliably pick and release caps without slippage or servo stalling.

Stage 4: Controller Link

NEXT

Connect Teensy Serial1 to Mega Serial1 via logic shifter. Test framed packet batch transfers and simulate cable loss.

Milestone: Mega completes in-flight cap safely when UART connection is severed.



Stage 5: Full Assembly

INTEGRATION

Integrate overhead camera gantry, LED diffuser plate, and sorting bins. Calibrate camera pixel to gripper tip offset (dx, dy).

Milestone: Fully autonomous multi-cap sort cycles with 100% binning accuracy.

Stage 6: Reliability Test

STRESS

Execute 100-cycle continuous batch sorting endurance run. Induce artificial faults: missed pickups, switch trips, E-Stop cuts.

Milestone: Zero system deadlocks, verified thermals, and safe fail-stop behavior.

Recommended Engineering Execution Order

Step 1: Confirm that the Teensy 4.1 receives and decodes raw QVGA frames from the ESP32-CAM reliably.
Step 2: Assemble the XY gantry and confirm TMC2208 stepper steps/mm calibration.
Step 3: Validate that the MG90S Z-axis vertical linkage reliably lifts the gripper assembly without gear slipping.
Step 4: Interconnect Teensy and Mega, validating the fail-safe communication recovery protocol under active motion.

10. Critical Engineering Considerations & Troubleshooting

Key failure modes, thermal limits, and electrical design rules that must be verified before commissioning hardware.

1. TMC2208 Vref Calculation & Thermals

The NEMA 17 motors are rated at 1.3 A per phase. Standard TMC2208 carrier boards with small heatsinks cannot sustain 1.3 A RMS continuously without triggering internal overtemperature thermal shutdown (T_J > 150^°C).

```
I_RMS = V_ref / (1.414 * R_sense)  [For R_sense = 0.11 Ω]
Set V_ref = 1.15 V  ⟶  I_RMS ≈ 0.74 A (Peak: 1.05 A)
Sufficient torque for gantry acceleration with safe thermal headroom.
```

2. Servo Power Isolation & Inrush Currents

Never power servos from the Arduino Mega 5V regulator pin! The onboard linear regulator can only supply ~500 mA before entering thermal shutdown. Two MG90S servos under active clamping draw transient stall current spikes up to 1.8 A. Powering them via the Mega will cause immediate brownouts and CPU reboot loops.

3. Ambient Lighting & Specular Glare

Plastic bottle caps have shiny, curved domes. Point-source overhead lighting causes bright white specular reflection spots where saturation drops to 0 (S ≈ 0), blinding the HSV thresholding algorithm. A diffused LED light ring or panel mounted overhead guarantees constant, shadow-free illumination regardless of ambient room daylight.

4. Z-Axis Linkage Counterbalance Spring

The single MG90S servo lifts the vertical slider, the parallel gripper linkage, the clamping servo, and the cap itself (~80–110 grams total). An auxiliary tension spring or counterweight should be installed on the vertical slider to balance the deadweight. This reduces continuous servo motor holding torque and eliminates gear tooth stripping.

5. Three Distinct Coordinate Reference Frames

The system contains three separate spatial frames: **(1) Camera Image Plane** (u, v) in pixels; **(2) Physical Sorting Board** (X_b, Y_b) in millimeters; **(3) Mechanical Toolhead Carriage** (X_g, Y_g) in stepper motor steps. The carriage homing offset and gripper tip offset vector (Δ X, Δ Y) must be calibrated independently of the camera homography matrix.

6. Missed Pickup & Jam Detection

A commanded servo grip does not guarantee that a cap was successfully grasped. If a cap is missing or deformed, the gripper closes to its minimum stop (<24 mm). Adding a micro optical presence switch or monitoring servo horn deflection prevents the robot from carrying "air" to the sorting bin.

SYMPTOM	PROBABLE ROOT CAUSE	ENGINEERING REMEDIATION
Teensy reboots or halts during image transfer	UART baud rate timing mismatch or buffer overflow	Verify Serial2 buffer is enlarged via `SERIAL2_RX_BUFFER_SIZE 1024`. Ensure baud rate is locked to 921,600 baud.
Mega resets when gripper clamps cap	Servo inrush current causing logic voltage dip	Ensure Branch B (Servos) is powered via dedicated 5V 5A buck and decoupling capacitor (1000\,µF) is on PCA9685 V+.
Stepper motor stutters or loses steps mid-travel	TMC2208 overheating or acceleration set too high	Lower acceleration from 2000 to 1200 mm/s^2. Verify driver heatsink airflow and adjust V_{ref} potentiometer.
Red caps detected as yellow or missing	Hue circle wrap-around threshold error	Red spans both ends of the hue spectrum. Must verify both slices: (H < 10 or H > 170).

11. Core Embedded Firmware Architecture (Part 1)

Complete production C++ firmware implementations for inter-controller wire packet structures and the ESP32-CAM frame acquisition streamer.

packet_protocol.h — Wire Protocol

HEADER

```
// packet_protocol.h - Inter-Controller Definitions
#ifndef PACKET_PROTOCOL_H
#define PACKET_PROTOCOL_H

#define PROTOCOL_MAGIC_BYTE   0x5A
#define CMD_CAPTURE_FRAME     0xAA
#define CMD_BATCH_START       0xB1
#define CMD_EXECUTE_PICK      0xC2
#define CMD_RECOVERY_HALT     0xEE

enum CapColor : uint8_t {
  COLOR_RED    = 0x01,
  COLOR_YELLOW = 0x02,
  COLOR_GREEN  = 0x03,
  COLOR_BLUE   = 0x04
};

#pragma pack(push, 1)
struct CapTarget {
  uint8_t  capId;
  CapColor color;
  uint16_t pixelX;
  uint16_t pixelY;
  float    boardX_mm;
  float    boardY_mm;
  uint8_t  targetBinId;
};

struct BatchHeader {
  uint8_t  magic;
  uint16_t batchId;
  uint8_t  totalCaps;
  uint16_t crc16;
};
#pragma pack(pop)

#endif
```

esp32cam_uart.ino — Frame Streamer

ESP32-CAM

```
// esp32cam_uart.ino - High-Speed Frame Streamer
#include "esp_camera.h"
#include <HardwareSerial.h>

HardwareSerial TeensySerial(2);

void setup() {
  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer   = LEDC_TIMER_0;
  config.pin_d0 = 5;  config.pin_d1 = 18;
  config.pin_d2 = 19; config.pin_d3 = 21;
  config.pin_d4 = 36; config.pin_d5 = 39;
  config.pin_d6 = 34; config.pin_d7 = 35;
  config.pin_xclk     = 0;  config.pin_pclk     = 22;
  config.pin_vsync    = 25; config.pin_href     = 23;
  config.pin_sscb_sda = 26; config.pin_sscb_scl = 27;
  config.pin_pwdn     = 32; config.pin_reset    = -1;
  config.xclk_freq_hz = 20000000;
  config.pixel_format = PIXFORMAT_RGB565;
  config.frame_size   = FRAMESIZE_QVGA; // 320x240
  config.fb_count     = 2;

  esp_camera_init(&config);
  TeensySerial.begin(921600, SERIAL_8N1, 14, 13);
}

void loop() {
  if (TeensySerial.available() &&
      TeensySerial.read() == 0xAA) {
    camera_fb_t *fb = esp_camera_fb_get();
    if (fb) {
      TeensySerial.write(0x55);
      TeensySerial.write(0xAA);
      TeensySerial.write((fb->len >> 8) & 0xFF);
      TeensySerial.write(fb->len & 0xFF);
      TeensySerial.write(fb->buf, fb->len);
      esp_camera_fb_return(fb);
    }
  }
}
```

11. Core Embedded Firmware Architecture (Part 2)

High-speed embedded image analytics running on Teensy 4.1 alongside non-blocking stepper pulse sequencing on Arduino Mega.

teensy_vision.cpp — Vision Planner

TEENSY 4.1

```cpp
// teensy_vision.cpp - Cortex-M7 @ 600 MHz
#include <Arduino.h>
#include "packet_protocol.h"

#define FRAME_BYTES (320 * 240 * 2) // 150 KiB
uint8_t imageBuffer[FRAME_BYTES];
CapTarget detectedCaps[100];
uint8_t capCount = 0;

float H[9] = {
  0.9854f, -0.0120f, -12.45f,
  0.0105f,  0.9782f, -14.10f,
  0.00004f, 0.00003f, 1.0000f
};

void transformToMillimeters(CapTarget* cap) {
  float u = cap->pixelX, v = cap->pixelY;
  float w = H[6]*u + H[7]*v + H[8];
  cap->boardX_mm = (H[0]*u + H[1]*v + H[2]) / w;
  cap->boardY_mm = (H[3]*u + H[4]*v + H[5]) / w;
}

void setup() {
  Serial.begin(115200);
  Serial1.begin(115200); // Mega Link
  Serial2.begin(921600); // ESP32 Link
}

void loop() {
  Serial2.write(CMD_CAPTURE_FRAME);
  if (receiveFrameWithCRC(imageBuffer, FRAME_BYTES)) {
    capCount = 0;
    segmentCapsHSV(imageBuffer, detectedCaps, &capCount);
    for (uint8_t i = 0; i < capCount; i++) {
      transformToMillimeters(&detectedCaps[i]);
    }
    optimizeSortingPath(detectedCaps, capCount);
    sendPlanBatchToMega(detectedCaps, capCount);
  }
  delay(100);
}
```

mega_motion.ino — Motion Sequencer

ARDUINO MEGA

```cpp
// mega_motion.ino - ATmega2560 @ 16 MHz
#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>
#include <AccelStepper.h>
#include "packet_protocol.h"

AccelStepper stepperX(AccelStepper::DRIVER, 2, 5);
AccelStepper stepperY(AccelStepper::DRIVER, 3, 6);
Adafruit_PWMServoDriver pca = Adafruit_PWMServoDriver(0x40);

enum RobotState {
  STATE_IDLE, STATE_HOMING, STATE_MOVING,
  STATE_LOWER_Z, STATE_GRIP, STATE_LIFT,
  STATE_TRAVERSE, STATE_RELEASE, STATE_RETRACT,
  STATE_FAILSAFE
};
RobotState currentState = STATE_HOMING;
CapTarget currentCap;
bool hasInFlightCap = false;

void setup() {
  Serial.begin(115200); Serial1.begin(115200);
  pca.begin(); pca.setPWMFreq(50);
  stepperX.setMaxSpeed(4000);
  stepperX.setAcceleration(2000);
  stepperY.setMaxSpeed(4000);
  stepperY.setAcceleration(2000);
  homeAxes();
}

void loop() {
  stepperX.run(); stepperY.run();
  executeMotionStateMachine();
  if (checkCommunicationLoss() && hasInFlightCap) {
    executeSafeLocalizedCompletion(); // FAIL-SAFE
  }
}
```

Engineering Verification & Project Sign-Off

APPROVED FOR PRODUCTION

Document Title: Autonomous Bottle-Cap Sorting Machine Technical Dossier
Authored & Engineered by: Neil Shettigar, Kahan, and Shourya
Principal Architecture, Firmware & Design: Neil Shettigar
Hardware Revisions: Teensy 4.1 Rev B • Arduino Mega 2560 R3 • ESP32-CAM AI-Thinker

GitHub Code Repository: github.com/Neilboss/CapSorter
Interactive Web Application: neilboss.github.io/CapSorter
Operational Safety Status: Fully Isolated Rails with Master Hardware E-Stop
Release Date & Timestamp: November 2026 • Production Release